

CS 473 (Spring 2025)

Homework 1 (due Feb 6 Thu 10am)

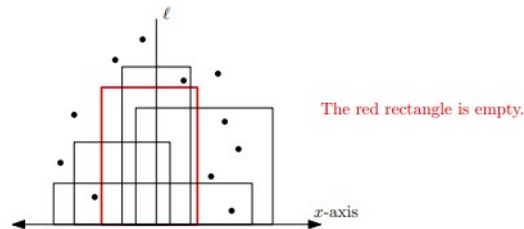
Instructions: Carefully read <http://courses.grainger.illinois.edu/cs473/sp2025/policies.html#hw> and <http://courses.grainger.illinois.edu/cs473/sp2025/integrity.html>. In particular, note the following:

- **Groups of up to three people may submit joint solutions.** Each problem should be submitted by exactly one person, and the beginning of the homework should clearly state the Gradescope names and email addresses of each group member. In addition, whoever submits the homework must tell Gradescope who their other group members are.
 - **Submit your solutions electronically on the course Gradescope site as PDF files.** Submit a separate PDF file for each numbered problem. If you plan to typeset your solutions (which we highly recommend), you may use the $\text{\LaTeX}$ solution template on the course web site. If you must submit scanned handwritten solutions, we recommend that you use a black pen on blank white paper and a high-quality scanner app (or an actual scanner, not just a phone camera).
 - You may use any source at your disposal—paper, electronic, or human—but you ***must cite every source that you use, and you must write everything yourself in your own words*** (see the course web pages).
 - Your solution will be judged not only for correctness but also for clarity and style (again, see the course web pages).
-

Problem 1.1: A rectangle r is called *grounded* if the bottom edge of r lies on the x -axis (thus, a grounded rectangle can be specified by 2 x -coordinates and 1 y -coordinate).

Given a set R of grounded rectangles and a set P of points in 2D with $n = |R| + |P|$, we want to find an *empty* rectangle $r \in R$, i.e., a rectangle that does not have any point of P inside, if report that none exists.

- (a) (50 pts) Consider the special case when all rectangles of R intersect a given vertical line ℓ . Design and analyze an $O(n \log n)$ -time algorithm to solve this special case. [Hint: don't use divide-and-conquer here; solve the problem directly by examining the points and rectangles in increasing y -order. You may assume no two points have the same x - or y -coordinates.]



Solution:

Define a list of all rectangles R , represented by (y, x_1, x_2) .

Define a list of all points P , represented by (y, x) .

Merge P and R into I , sort according to y -coord:

$O(n \log n)$

```
// Ensure points are processed before rectangles
I = mergeSort(R + P)
```

Maintain two points p_l, p_r that are closest to the line, one on each side.

```
p_l, p_r = -inf, inf
```

Iterate through I . For each point, check if it is closer to ℓ ;

For each rectangle, check if p_l or p_r is inside:

$O(n)$

```
for i=0 to I.length:
    if isPoint(I[i]):
        x = I[i][1] // Get the x-coord of the point
        if x == l: // No solution if it is on the line
            return None
        // check if it is closer to the line
        if p_l < x < l:
            p_l = x
```

```

        if l < x < p_r:
            p_r = x
    else:
        // Get the x-coord of the rectangle
        x_1, x_2 = I[i][1], I[i][2]
        if p_l < x_1 and p_r > x_2: // no points inside
            return I[i]

```

Total time complexity is $O(n \log n)$.

- (b) (50 pts) Design and analyze an efficient algorithm to solve the original problem (without the restriction about ℓ), by using divide-and-conquer (and using part (a) as a subroutine).

Solution:

Define a list of all rectangles R , rectangles are represented as (y, x_1, x_2) .

Define a list of all points P , points are represented as (y, x) .

Merge R and P into I , sort according to x_1 of rectangles and x of points: $O(n \log n)$

```

I = mergeSort(R + P)
S = I // current subarea

```

For each subarea S , get lists of rectangles r and points p for subroutine.

Meanwhile, maintain a boolean list b to keep track of rectangles that are crossed by the vertical line. $O(n)$ in total

```

l = (S[0] + S[S.length - 1]) / 2 // vertical line
for i = 0 to S.length:
    if isPoint(S[i]):
        p.add(S[i])
    else:
        r.add(S[i])
        if S[i][1] < l < S[i][2]: // cross the vertical line
            b[i] = True
if r.length == 0: // no rectangle in the subarea
    return

```

With all rectangles that are crossed with the line and all points in the subarea, call solution in part(a). $O(n \log^2 n)$ in total

If there is no empty rectangles, split the subarea into 2 parts by the vertical line, each containing only uncrossed rectangles and points. Recursively process each subarea until an empty rectangle is found.

Total time complexity is $O(n \log^2 n)$.

Problem 1.2: Given three sets of integers A , B , and C with a total size $|A| + |B| + |C| = n$, we want to decide whether there exist elements $a \in A$, $b \in B$, and $c \in C$ such that $c = a + b$. It is not difficult to obtain a near $O(n^2)$ time algorithm for this problem. Here, we will explore subquadratic algorithms for some interesting special cases.

- (a) (20 pts) Give an algorithm that runs in $O(|A||B|\log n)$ time, assuming that C is stored in a sorted array.

Solution:

Iterate through all integers in A and B , compute $s = a_i + b_j$. Then check if s is in C using binary search.

```
for i = 0 to A.length:
    for j = 0 to B.length: // O(|A||B|)
        s = A[i] + B[j]
        if binarySearch(C, s): // O(log n)
            return True
```

Total time complexity is $O(|A||B|\log n)$.

- (b) (40 pts) For the case when $A, B, C \subset \{1, \dots, n^{1.5}\}$, give an algorithm that runs in $O(n^{1.5} \log n)$ time, by using polynomial multiplication or convolution (on vectors of length $n^{1.5}$).

Solution:

Define an array P of size $O(n^{1.5})$ initialized to zero. For each element $a \in A$, set $P[a] = 1$. Define an array Q of size $O(n^{1.5})$ initialized to zero. For each element $b \in B$, set $Q[b] = 1$. Compute the Discrete Fourier Transform of P and Q using Fast Fourier Transform, $P' = \text{FFT}(P)$, $Q' = \text{FFT}(Q)$ in $O(n^{1.5} \log n)$ time.

Multiply the transformed arrays element-wise, $R' = P' \cdot Q'$ in $O(n^{1.5})$ time.

Compute the Inverse FFT to obtain the convolution result, $R = \text{IFFT}(R')$. Round the values in R to the nearest integers.

For each element $c \in C$, check if $R[c] > 0$ in $O(n^{1.5})$ time:

If any such c exists, return True. Otherwise, return False.

Total time complexity is $O(n^{1.5} \log n)$.

- (c) (40 pts) For the case when $C \subset \{1, \dots, n^{1.5}\}$ and A and B are arbitrary sets of integers (with no restrictions on the range), give an algorithm that runs in $O(n^t)$ time for some constant t strictly smaller than 2.

[Hint: for A and B , divide into intervals of length $n^{1.5}$. For intervals with fewer than r elements, use the algorithm in (a). For intervals with more than r elements (how many such intervals can there be?), use the algorithm in (b). How should we set the parameter r ?]

Problem 1.3:

- (a) (20 pts) Show that the product of an $n \times \ell$ with an $\ell \times n$ matrix can be computed in $O(\ell^{0.81}n^2)$ time by using Strassen's algorithm as a subroutine.

Solution:

Viewing A as n/ℓ row blocks of size $\ell \times \ell$ and B as n/ℓ column blocks of size $\ell \times \ell$, each multiplication involves two $\ell \times \ell$ matrix multiplication, Strassen's algorithm computes the product of two $m \times m$ matrices in $O(m^{2.81})$ time, so the total complexity is $O(\ell^{2.81}(n/\ell)^2) = O(\ell^{0.81}n^2)$.

- (b) (80 pts) We are given n strings $s^{(1)}, \dots, s^{(n)} \in (\Sigma \cup \{?\})^*$ where $?$ is the "don't care" symbol. Each string $s^{(i)}$ has length exactly ℓ .

We want to determine whether there exist two strings $s^{(i)}$ and $s^{(j)}$ ($i \neq j$) that *match*. Here, two strings $s^{(i)} = s_1^{(i)} s_2^{(i)} \dots s_\ell^{(i)}$ and $s^{(j)} = s_1^{(j)} s_2^{(j)} \dots s_\ell^{(j)}$ in $(\Sigma \cup \{?\})^*$ are said to match iff for all $k \in \{1, \dots, \ell\}$, we have $s_k^{(i)} = s_k^{(j)}$ or $s_k^{(i)} = ?$ or $s_k^{(j)} = ?$.

The trivial algorithm runs in $O(\ell n^2)$ time. Describe a faster algorithm.

[Hint: use the idea from Clifford and Clifford's algorithm, but replace convolution with matrix multiplication...]

Solution:

For each symbol $a \in \Sigma$, define the following two $n \times \ell$ $\{0, 1\}$ -matrices:

$$X_a(i, k) = \begin{cases} 1, & \text{if } s_k^{(i)} = a, \\ 0, & \text{otherwise,} \end{cases}$$

and

$$Z_a(i, k) = \begin{cases} 1, & \text{if } s_k^{(i)} \in \Sigma \setminus \{a\}, \\ 0, & \text{if } s_k^{(i)} = a \text{ or } s_k^{(i)} = ?. \end{cases}$$

The idea is that if at some position k we have $s_k^{(i)} = a$ and $s_k^{(j)} \neq a$ (and in particular, $s_k^{(j)} \in \Sigma$, not $?$) then the pair (i, j) cannot match.

For each $a \in \Sigma$, compute the $n \times n$ matrix

$$M_a = X_a \cdot (Z_a)^T,$$

where entry (i, j) in M_a counts the number of positions k such that $s_k^{(i)} = a$ and $s_k^{(j)} \in \Sigma \setminus \{a\}$.

Then, define

$$M(i, j) = \sum_{a \in \Sigma} M_a(i, j).$$

Notice that if $M(i, j) > 0$, then there exists at least one position where $s^{(i)}$ and $s^{(j)}$ conflict (i.e., they both have a letter from Σ but the letters differ). Therefore, if $M(i, j) = 0$ for some $i \neq j$, then for every position k either $s_k^{(i)} = s_k^{(j)}$, or at least one of $s_k^{(i)}$ and $s_k^{(j)}$ is $?$; that is,

$s^{(i)}$ and $s^{(j)}$ match.

Each multiplication $X_a \cdot (Z_a)^T$ is the product of an $n \times \ell$ and an $\ell \times n$ matrix. By part (a) (using Strassen's algorithm as a subroutine), each such product can be computed in $O(\ell^{0.81}n^2)$ time. Since there are only $|\Sigma|$ symbols (and typically $|\Sigma|$ is a constant), the total time is $O(\ell^{0.81}n^2)$.